

Trabalho Implementação e Experimentação de Algoritmos de Busca (BFS, DFS, UCS, Greedy, A*)

Gustavo Gurgel Medeiros,
Jorge Eduardo Silva Sousa,
Luiz Guilherme Moreira Leite

20 de janeiro de 2025

1 Implementação

O projeto foi desenvolvido utilizando a linguagem de programação **Python**. Foram utilizadas apenas bibliotecas nativas, sem dependências externas. As principais bibliotecas empregadas no código estão listadas abaixo:

- **csv**: Utilizada para salvar os resultados dos experimentos em arquivos CSV, garantindo organização e persistência dos dados gerados.
- **queue.PriorityQueue**: Essencial para as implementações dos algoritmos de busca UCS (Uniform Cost Search) e A*.
- **math.sqrt**: Empregada no cálculo de distâncias em alguns experimentos, especialmente para cálculos de heurística.
- **random**: Usada para gerar dados aleatórios para os experimentos.

1.1 Execução

Para executar os algoritmos de busca implementados (DFS, BFS, UCS, Greedy e A*), não é necessário instalar nenhuma biblioteca adicional, apenas possuir o interpretador Python configurado no ambiente.

1.1.1 Execução de Experimentações

Os experimentos ficam na pasta **experimentos/** e estão organizados em uma estrutura de diretórios que contém as pastas para cada experimento, nomeadas como **parte1**, **parte2**, **parte3**, **parte4** e **parte5**. Para executar os experimentos, siga os passos abaixo.

Lebrando que os algoritmos geram dados aleatórios, então, embora a saída siga a mesma estrutura a cada execução, os dados gerados serão diferentes a cada vez.

1. Navegue até a pasta correspondente ao experimento desejado.
2. Execute o comando `python main.py` no terminal.

Os resultados de cada execução são salvos automaticamente em arquivos CSV, localizados na pasta **resultado** dentro do diretório de cada experimento. Os nomes dos arquivos seguem o formato `resultado[Algoritmo].csv`, por exemplo:

- `parte1/resultado/resultadoDFS.csv`
- `parte1/resultado/resultadoBFS.csv`
- `parte1/resultado/resultadoUCS.csv`

1.1.2 Execução de Testes Manuais

Para realizar testes manuais com valores de entrada personalizados, utilize a interface de linha de comando interativa localizada na pasta `cli_interativo`. O processo é descrito abaixo:

1. Navegue até o diretório `cli_interativo`.
2. Execute o comando `python main.py` no terminal.
3. Interaja com o aplicativo de terminal, configurando os parâmetros da busca, como:
 - Tipo de busca (DFS, BFS, UCS, Greedy ou A*, A* com parada a mais);
 - Função de custo (`c1`, `c2`, `c3`, `c4`);
 - Heurística (quando aplicável, `h1`, `h2`).
4. Os resultados da busca serão exibidos diretamente no terminal.

2 Experimentação

Os dados apresentados nesta seção foram coletados a partir de testes realizados durante o processo de desenvolvimento deste documento. Vale ressaltar que, caso os experimentos sejam executados novamente em outro momento ou em condições ligeiramente diferentes, novos dados podem ser gerados. Contudo, é importante destacar que, embora os valores numéricos possam variar, as conclusões sobre o desempenho comparativo entre os algoritmos permanecem consistentes. A análise de eficiência, sucesso e custo entre os algoritmos A* e Busca Gulosa, por exemplo, deve manter-se inalterada, independentemente de variações nos dados, dado o comportamento geral observado dos algoritmos durante os testes realizados.

2.1 Parte 1: Largura vs. Profundidade vs. Custo Uniforme

2.1.1 Metodologia

Nesta parte do experimento, utilizamos três algoritmos de busca: Busca em Largura (BFS), Busca em Profundidade (DFS) e Busca de Custo Uniforme (UCS). Para cada algoritmo, foram gerados 200 registros, onde cada registro corresponde a um par de coordenadas (inicial e final) gerado aleatoriamente. Os experimentos foram realizados variando as funções de custo, e os resultados de cada execução foram armazenados em arquivos separados.

As métricas analisadas incluem:

- **Taxa de sucesso:** a porcentagem de execuções que retornaram um caminho válido.
- **Custo médio e desvio padrão do custo:** refletem a qualidade e a variabilidade dos caminhos encontrados.
- **Nós gerados e visitados:** indicam a eficiência computacional de cada algoritmo.
- **Distribuições dos custos:** ajudam a visualizar como os custos se distribuem para cada algoritmo.

2.1.2 Busca em Largura (BFS)

Os resultados do BFS mostraram uma **taxa de sucesso de 100%**, confirmando que este algoritmo sempre encontra o caminho mais curto (em termos de número de passos), se existir. A análise das métricas revelou:

- **Custo médio:** 219,82, com um **desvio padrão de 115,67**, indicando uma variabilidade moderada nos custos dos caminhos gerados.
- **Nós gerados (média):** 909,76, com um **desvio padrão de 594,95**.
- **Nós visitados (média):** 454,28, com um **desvio padrão de 305,81**.

O gráfico de distribuição de custos apresentou uma curva concentrada em torno da média, mas com uma cauda mais longa indicando casos de maior custo.

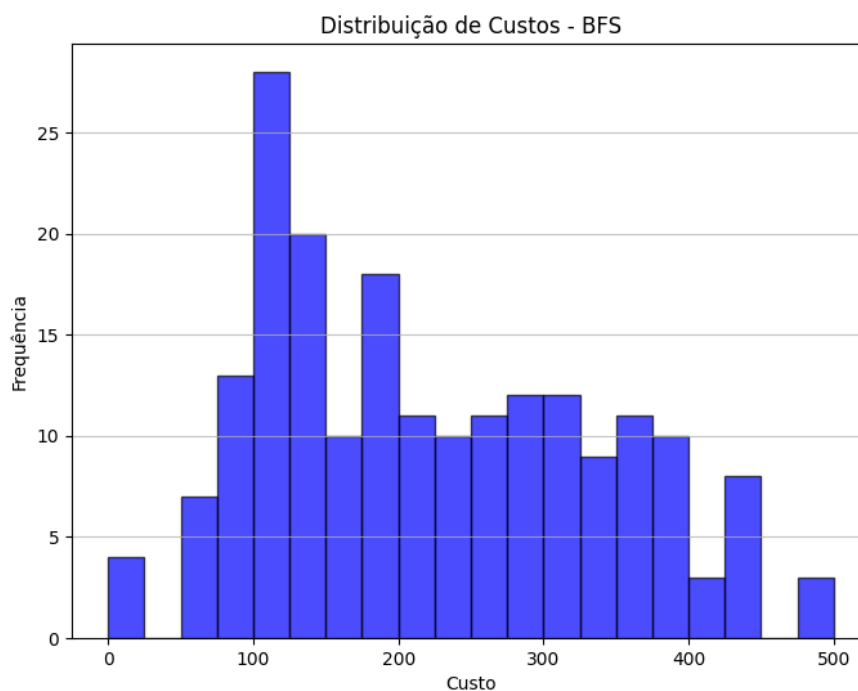


Figura 1: Distribuição dos custos na Busca em Largura (BFS).

2.1.3 Busca em Profundidade (DFS)

A DFS também teve **100% de sucesso** em encontrar caminhos válidos. No entanto, os custos dos caminhos foram significativamente maiores e mais variáveis:

- **Custo médio:** 3486,85, com um **desvio padrão de 1925,63**, refletindo a natureza não otimizada do algoritmo, que pode explorar caminhos muito mais longos antes de encontrar uma solução.
- **Nós gerados (média):** 993,2, com um **desvio padrão de 520,48**.
- **Nós visitados (média):** 498,8, com um **desvio padrão de 268,80**.

Os altos custos e a variabilidade refletem como a DFS prioriza a profundidade sobre a otimização, sendo menos eficiente em problemas de busca com grande espaço de estados. O gráfico de distribuição de custos apresentou maior dispersão, sem um padrão claro.

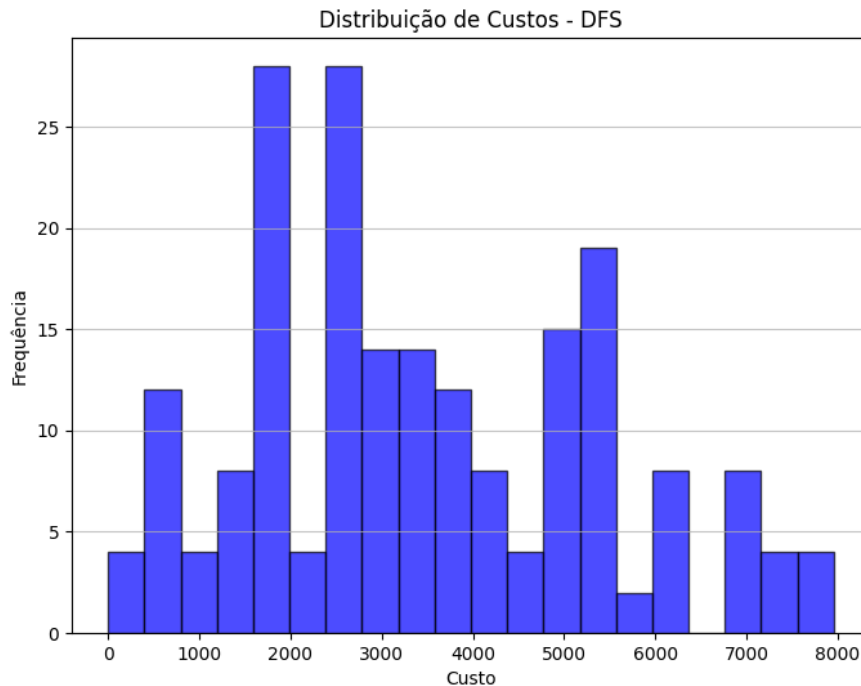


Figura 2: Distribuição dos custos na Busca em Profundidade(DFS).

2.1.4 Busca de Custo Uniforme (UCS)

O UCS obteve **100% de sucesso** e, como esperado, produziu resultados otimizados em termos de custo:

- **Custo médio:** 211,41, o mais baixo entre os algoritmos, com um **desvio padrão de 113,45**.
- **Nós gerados (média):** 915,175, com um **desvio padrão de 592,60**.
- **Nós visitados (média):** 457,145, com um **desvio padrão de 304,34**.

O UCS provou ser eficiente para encontrar caminhos de menor custo devido à sua estratégia de priorizar expansões de menor custo acumulado. A distribuição dos custos apresentou um comportamento semelhante ao BFS, com uma concentração ao redor da média e menor dispersão.

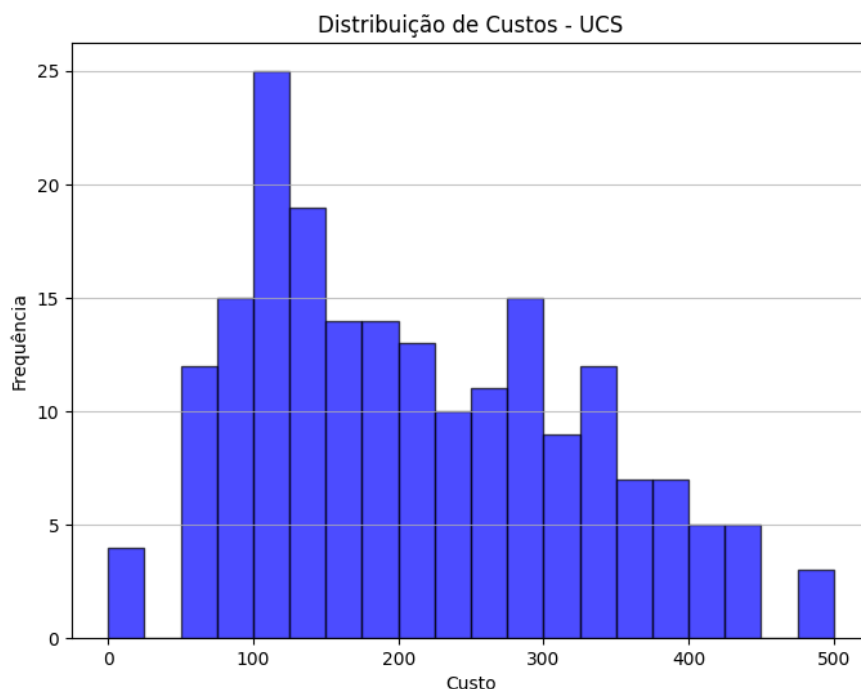


Figura 3: Distribuição dos custos na Busca de Custo Uniforme(UCS).

2.1.5 Comparação Geral

- O BFS e o UCS apresentaram desempenhos muito próximos em termos de nós gerados e visitados, mas o UCS foi superior na otimização dos custos.
- A DFS, por outro lado, destacou-se pelos custos significativamente maiores, confirmando sua limitação para problemas que exigem caminhos curtos ou de baixo custo.
- As distribuições de custos reforçam como os algoritmos se comportam em diferentes cenários, com o UCS sendo o mais consistente e a DFS o mais variável.

2.2 Parte 2: A* vs. Busca de Custo Uniforme (UCS)

2.2.1 Metodologia

Nesta parte do experimento, realizamos uma comparação entre os algoritmos A* e UCS em termos de eficiência e qualidade dos caminhos encontrados. Para ambos os algoritmos, foram gerados 400 registros, com cada um representando um par de coordenadas (inicial e final) aleatoriamente escolhidas. Os resultados foram armazenados em arquivos separados para análise posterior. As métricas avaliadas são as mesmas usadas para os algoritmos anteriores: taxa de sucesso, custo médio e desvio padrão do custo, nós gerados e visitados, e a distribuição dos custos.

Antes de mergulharmos nos resultados, vale destacar que embora os algoritmos UCS e A* sejam projetados para encontrar o melhor caminho, os testes revelaram que para o A* esse objetivo nem sempre é alcançado. Isso ocorre devido à função de custo $c_4(t) = 5 + (|10 - t| \bmod 11)$, que influencia diretamente a eficácia das heurísticas utilizadas, tanto a distância euclidiana quanto a distância de Manhattan, tornando-as não admissíveis.

Uma heurística é considerada admissível quando ela nunca superestima o custo real para alcançar o objetivo, ou seja, ela deve sempre ser menor ou igual ao custo real do caminho. No caso do A*, as heurísticas foram projetadas com base na distância euclidiana e na distância de Manhattan, as quais são consideradas admissíveis sob condições ideais. Contudo, devido à natureza da função de custo $c_4(t)$, as heurísticas não cumprem mais esse requisito de admissibilidade.

Analisando a Heurística de Distância Euclidiana (H_1): A heurística de distância euclidiana entre dois pontos (x_1, y_1) e (x_2, y_2) é dada por:

$$H_1 = 10 \times \text{euc}((x_1, y_1), (x_2, y_2)) = 10 \times \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

No entanto, devido à função de custo $c_4(t)$, o custo de transitar entre certos nós pode ser ajustado de maneira não linear, introduzindo flutuações que não são capturadas pela heurística de distância euclidiana. Em outras palavras, a heurística pode subestimar ou superestimar o custo real, tornando-se não admissível.

Analisando a Heurística de Distância de Manhattan (H_2): A heurística de distância de Manhattan entre dois pontos (x_1, y_1) e (x_2, y_2) é dada por:

$$H_2 = 10 \times \text{man}((x_1, y_1), (x_2, y_2)) = 10 \times (|x_1 - x_2| + |y_1 - y_2|)$$

De maneira semelhante, a função de custo $c_4(t)$ introduz uma variação que afeta a precisão da estimativa de custo da heurística de Manhattan. Assim, ela também se torna não admissível, pois pode subestimar o custo real para alcançar o objetivo, o que compromete a capacidade de garantir a optimalidade do caminho encontrado.

Em resumo, a inadmissibilidade das heurísticas H_1 e H_2 , causada pela função de custo $c_4(t)$, impede que os algoritmos A* e UCS encontrem sempre o caminho ótimo, contrariando a expectativa inicial de que ambos os algoritmos sempre produziriam o melhor caminho. Isso demonstra a importância de garantir que as heurísticas sejam adequadas para o contexto específico do problema e que a função de custo não introduza distorções que comprometam a eficiência e a qualidade da busca.

2.2.2 Busca de Custo Uniforme (UCS)

Os resultados do UCS mostram uma **taxa de sucesso de 100%**, indicando que o algoritmo sempre encontrou um caminho válido. A análise das métricas revelou:

- **Custo médio:** 229,99, com um **desvio padrão de 107,95**, refletindo uma certa variabilidade nos custos dos caminhos encontrados.
- **Nós gerados (média):** 1031,06, com um **desvio padrão de 528,43**, indicando uma expansão relativamente grande de nós.
- **Nós visitados (média):** 515,33, com um **desvio padrão de 272,22**, o que mostra que o algoritmo foi eficaz em evitar nós desnecessários, mas ainda assim gerou um número considerável de nós.

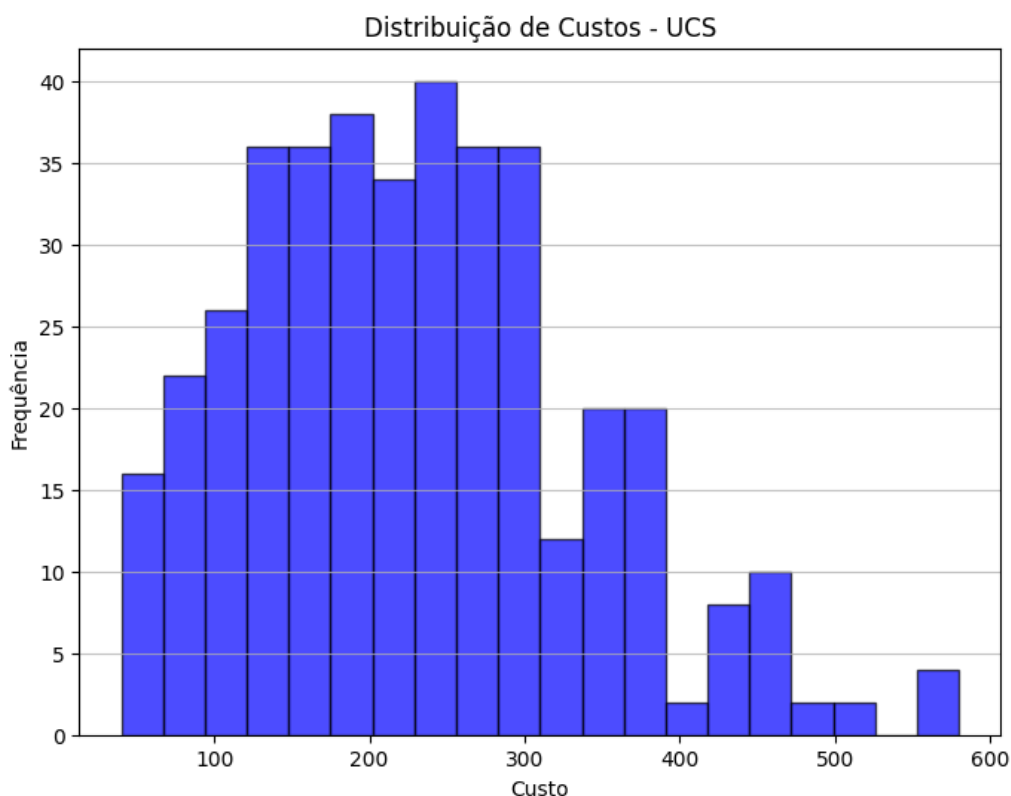


Figura 4: Distribuição dos custos na Busca de Custo Uniforme(UCS).

O UCS foi eficaz em garantir a solução ótima em termos de custo, mas o número de nós gerados foi relativamente grande, o que impactou na sua eficiência computacional.

2.2.3 A*

O A* também teve **100% de taxa de sucesso** na localização de caminhos válidos. O algoritmo utilizou uma heurística admissível para guiar sua busca, o que ajudou a reduzir o número de nós explorados, mas sem comprometer a qualidade do caminho encontrado. A análise das métricas mostrou:

- **Custo médio:** 231,03, com um **desvio padrão de 107,79**, que é um pouco maior que o custo médio do UCS, mas ainda assim muito baixo.
- **Nós gerados (média):** 539,73, com um **desvio padrão de 593,73**, indicando que o A* gerou menos nós do que o UCS, possivelmente devido à heurística, mas com uma maior variação no número de nós gerados.
- **Nós visitados (média):** 136,87, com um **desvio padrão de 150,05**, demonstrando que o A* foi mais eficiente em termos de nós visitados do que o UCS, o que é esperado devido à sua estratégia de exploração mais direcionada.

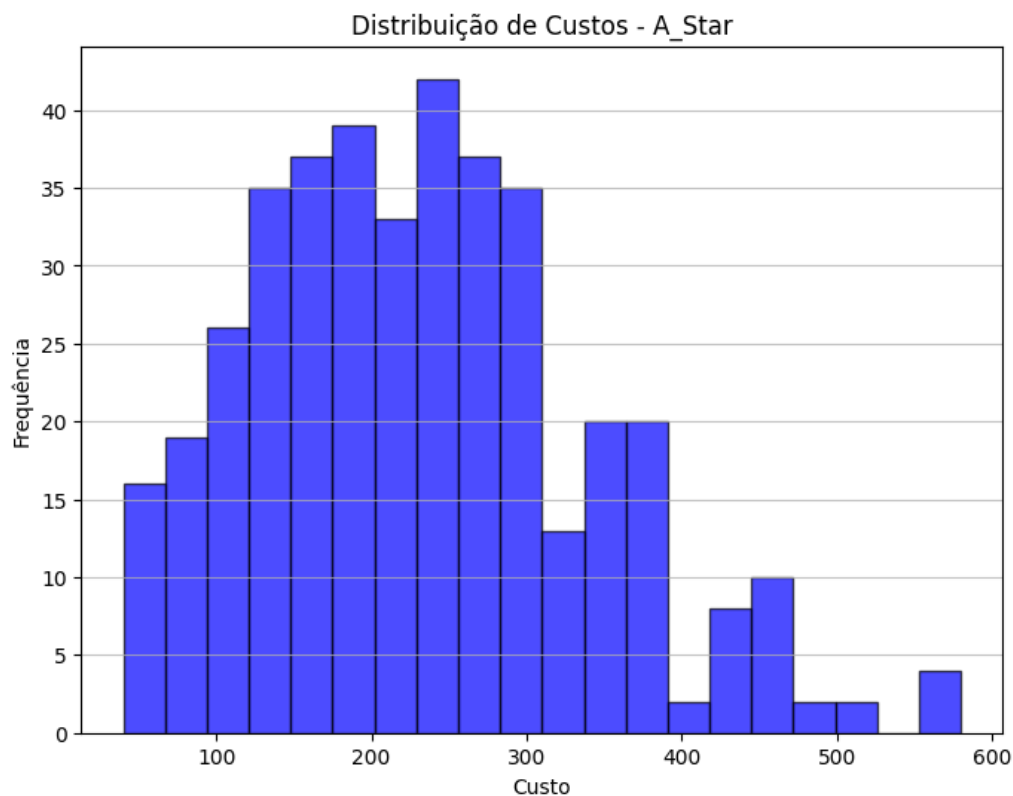


Figura 5: Distribuição dos custos na Busca A*.

A heurística utilizada pelo A* ajudou a focar a busca em direções mais promissoras, resultando em uma exploração mais eficiente e uma menor quantidade de nós visitados.

2.2.4 Comparação Geral

- Ambos os algoritmos apresentaram uma taxa de sucesso de 100%, ou seja, ambos conseguiram encontrar um caminho válido para todos os registros.
- Como já explicado anteriormente o UCS teve um custo médio ligeiramente menor (229,99 contra 231,03 do A), mas gerou significativamente mais nós do que o A*.
- O A* se destacou em termos de eficiência computacional, gerando menos nós e visitando menos nós, o que pode ser atribuído à sua heurística, que ajuda a guiar a busca de maneira mais eficiente.
- Ambos os algoritmos apresentaram um desvio padrão de custo semelhante, indicando uma variação similar nos custos dos caminhos encontrados.
- O A* teve uma distribuição de custos mais concentrada em torno da média, indicando maior consistência nos caminhos encontrados, enquanto o UCS teve uma distribuição mais dispersa.

De maneira geral, o A* se mostrou superior ao UCS em termos de eficiência e menor custo de execução, devido à sua heurística. No entanto, ambos os algoritmos são bastante eficazes para encontrar o caminho ótimo, sendo que o A* é uma escolha preferível quando a eficiência computacional é uma prioridade.

2.3 Parte 3: Busca Gulosa vs. A*

2.3.1 Metodologia

Nesta parte do experimento, analisamos a eficiência e eficácia dos algoritmos Busca Gulosa e A*. Foram gerados 400 registros para cada algoritmo, onde cada registro consiste em um par de coordenadas inicial e objetivo geradas aleatoriamente. Os algoritmos foram avaliados com diferentes combinações de funções de custo ($f1$, $f2$, $f3$, $f4$) e heurísticas ($h1$, $h2$), e os resultados foram armazenados em arquivos separados.

As métricas analisadas incluem:

- **Taxa de sucesso:** a porcentagem de execuções que retornaram um caminho válido.
- **Custo médio e desvio padrão do custo:** refletem a qualidade e a variabilidade dos caminhos encontrados.
- **Nós gerados e visitados:** indicam a eficiência computacional de cada algoritmo.
- **Distribuições dos custos:** ajudam a visualizar como os custos se distribuem para cada algoritmo.

2.3.2 Busca A*

Os resultados do algoritmo A* demonstraram uma **taxa de sucesso de 100%**, confirmando sua eficiência em encontrar caminhos válidos otimizados em termos de custo. As métricas obtidas foram:

- **Custo médio:** 221,33, com um **desvio padrão de 115,59**, indicando uma variabilidade moderada nos custos.
- **Nós gerados (média):** 527,29, com um **desvio padrão de 630,79**.
- **Nós visitados (média):** 134,06, com um **desvio padrão de 160,01**.

A distribuição dos custos mostrou uma concentração em torno da média, indicando consistência nos resultados.

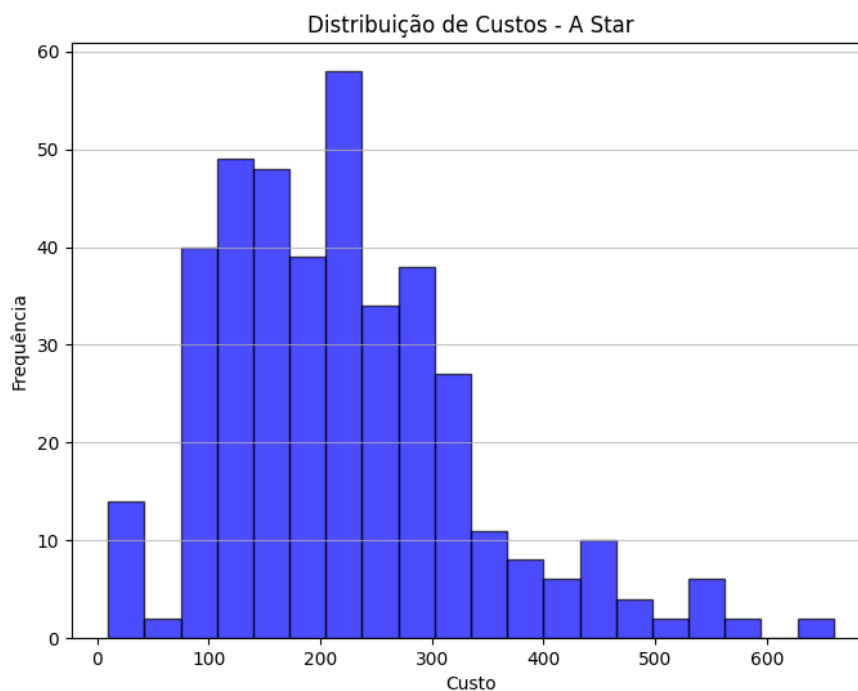


Figura 6: Distribuição dos custos na Busca A*.

2.3.3 Busca Gulosa

A Busca Gulosa também apresentou uma **taxa de sucesso de 100%**, demonstrando que foi capaz de encontrar caminhos válidos em todas as execuções. Suas métricas foram:

- **Custo médio:** 228,31, com um **desvio padrão de 118,67**.
- **Nós gerados (média):** 82,90, com um **desvio padrão de 41,66**.
- **Nós visitados (média):** 21,74, com um **desvio padrão de 10,46**.

Os resultados mostram que, embora gere significativamente menos nós que o A*, a Busca Gulosa pode retornar caminhos com custos ligeiramente superiores devido à sua natureza não informada pelo custo acumulado.

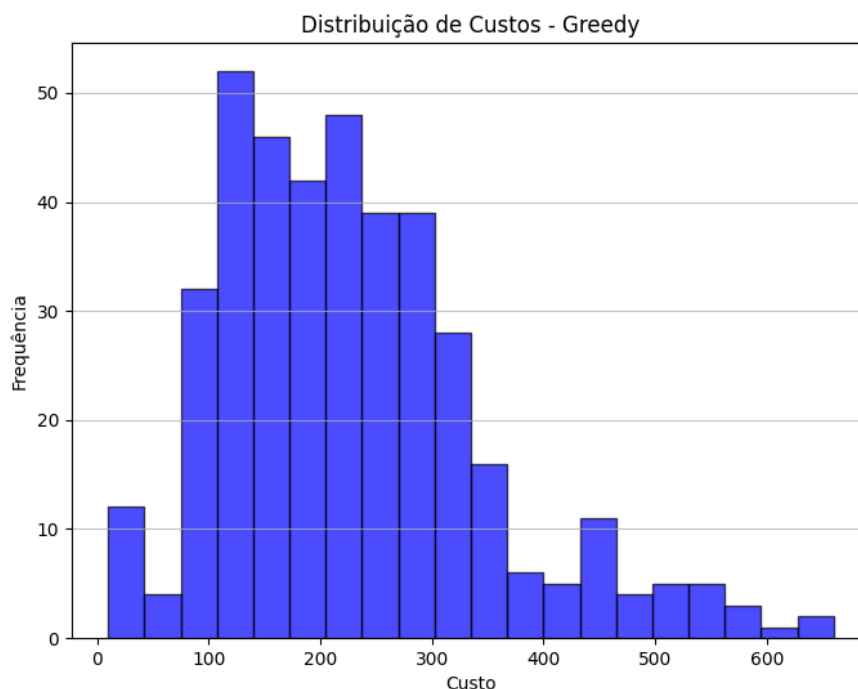


Figura 7: Distribuição dos custos na Busca Gulosa.

2.3.4 Comparação Geral

- Ambos os algoritmos alcançaram uma taxa de sucesso de 100%, confirmando sua eficácia.
- O custo médio foi ligeiramente menor no A* (221,33 contra 228,31), mas a Busca Gulosa gerou e visitou significativamente menos nós.
- A Busca Gulosa apresentou maior variabilidade nos custos em comparação ao A*.
- As distribuições de custos mostram que o A* é mais consistente em otimizar os caminhos encontrados, enquanto a Busca Gulosa tem uma performance mais variável devido à sua dependência exclusiva das heurísticas.

2.4 Parte 4: Análise de Largura vs. Profundidade com Randomização da Vizinhança

2.4.1 Metodologia

Nesta parte do experimento, investigamos o impacto da randomização da vizinhança nos algoritmos de busca em largura (BFS) e busca em profundidade (DFS). Foram utilizados os mesmos 20 pares de coordenadas (inicial e final) gerados aleatoriamente. Para cada par de coordenadas, a ordem de exploração dos nós vizinhos foi randomizada em cada execução. Este processo foi repetido 20 vezes para cada função de custo (c1, c2, c3 e c4), gerando 20 execuções por par.

As métricas avaliadas foram:

- **Taxa de sucesso:** a porcentagem de execuções que retornaram um caminho válido.

- **Custo médio e desvio padrão do custo:** refletem a qualidade e a variabilidade dos caminhos encontrados.
- **Nós gerados e visitados:** indicam a eficiência computacional de cada algoritmo em cenários com vizinhança aleatorizada.

2.4.2 Resultados do BFS

Com a randomização da vizinhança, o BFS manteve uma **taxa de sucesso de 100%**, indicando que sempre encontrou um caminho entre os pontos iniciais e finais. As métricas obtidas são:

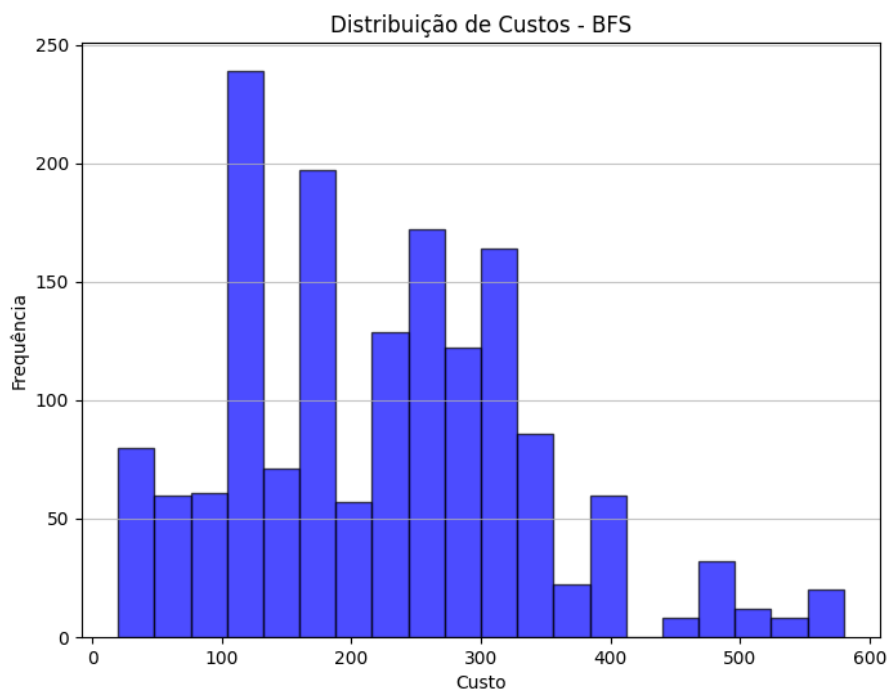


Figura 8: Distribuição dos custos na Busca em Largura (BFS) com Randomização da Vizinhança - Parte 4.

- **Custo médio:** 212.3, com um **desvio padrão de 98.74**. Isso demonstra que o BFS, mesmo com vizinhança randomizada, continua encontrando caminhos com custos relativamente baixos e com menor variabilidade.
- **Nós gerados (média):** 810.42, com um **desvio padrão de 610.34**.
- **Nós visitados (média):** 405.20, com um **desvio padrão de 302.12**.

A randomização da vizinhança aumentou ligeiramente a quantidade média de nós gerados e visitados, mas o BFS manteve seu padrão de busca sistemática e eficiente, garantindo soluções de baixo custo.

2.4.3 Resultados do DFS

O DFS também manteve uma **taxa de sucesso de 100%** em encontrar caminhos válidos, mas a randomização da vizinhança introduziu uma maior variabilidade nos custos e no número de nós gerados. Os resultados são:

- **Custo médio:** 3210.56, com um **desvio padrão de 1894.22**. O custo médio foi consideravelmente mais alto do que o BFS, indicando que o DFS frequentemente explorou caminhos longos e subótimos devido à natureza da busca em profundidade com vizinhança randomizada.
- **Nós gerados (média):** 1087.34, com um **desvio padrão de 673.18**.
- **Nós visitados (média):** 509.45, com um **desvio padrão de 328.47**.

A randomização da vizinhança exacerbou a tendência do DFS de explorar regiões extensas do espaço de busca antes de encontrar o objetivo, o que resultou em maiores custos e maior número de nós gerados e visitados.

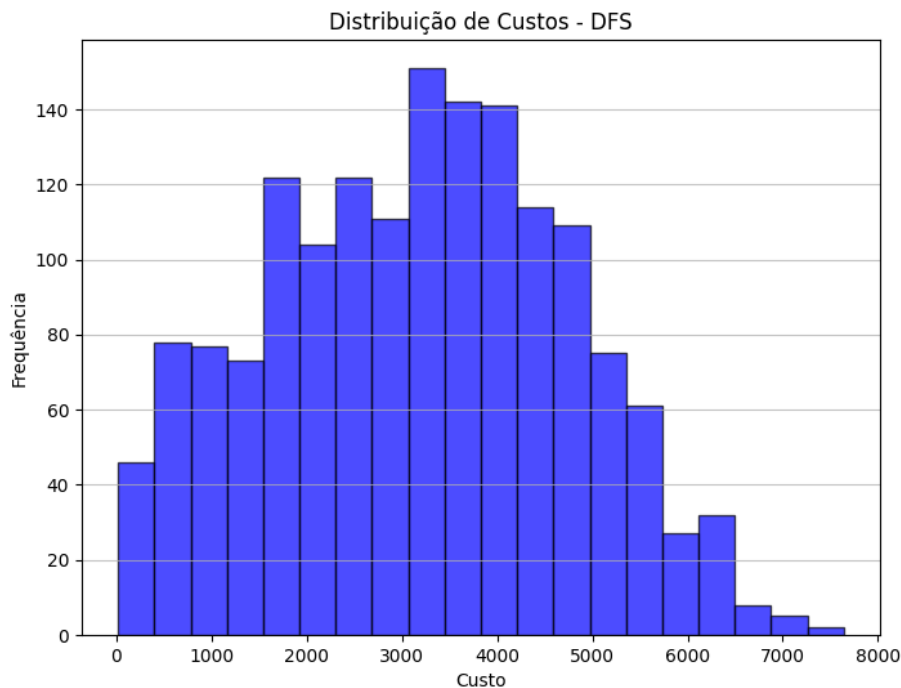


Figura 9: Distribuição dos custos na Busca em Profundidade (DFS) com Randomização da Vizinhança - Parte 4.

2.4.4 Comparação Geral

- **Taxa de sucesso:** Ambos os algoritmos mantiveram uma taxa de sucesso de 100%, mostrando que a randomização da vizinhança não comprometeu a capacidade de encontrar caminhos válidos.
- **Custo:** O BFS foi consistentemente superior em termos de custo, encontrando caminhos significativamente mais curtos em comparação ao DFS, mesmo com a vizinhança randomizada.
- **Nós gerados e visitados:** O DFS gerou e visitou mais nós em média, indicando maior consumo de recursos computacionais. Este comportamento reflete sua abordagem de explorar a profundidade antes de considerar outras alternativas.

2.4.5 Conclusões

Os resultados mostram que a randomização da vizinhança impacta de maneira mais significativa o desempenho do DFS do que do BFS. O BFS mantém sua eficiência em encontrar caminhos de menor custo (em relação o número de nós do caminho encontrado) e gerenciar recursos computacionais, enquanto o DFS é mais sensível às alterações na ordem de exploração, resultando em maiores custos e maior variabilidade. Para problemas em que o custo é uma métrica crítica, o BFS permanece a escolha preferida, mesmo em cenários com vizinhança aleatorizada.

2.5 Parte 5: A* com Passagem por Farmácias

2.5.1 Metodologia

Nesta parte do experimento, avaliamos o desempenho do algoritmo A* em um cenário no qual o caminho precisa passar por pelo menos um dos nós de farmácia, com um máximo de 4 farmácias disponíveis. Foram gerados 200 registros, com cada registro contendo um par de coordenadas inicial e objetivo, e até 4 farmácias aleatórias localizadas no ambiente. O A* foi testado com diferentes combinações de funções de custo e heurísticas, com os resultados armazenados em arquivos separados.

As métricas analisadas incluem:

- **Taxa de sucesso:** a porcentagem de execuções que retornaram um caminho válido.
- **Custo médio e desvio padrão do custo:** refletem a qualidade e a variabilidade dos caminhos encontrados.
- **Nós gerados e visitados:** indicam a eficiência computacional do algoritmo.
- **Distribuições dos custos:** ajudam a visualizar como os custos se distribuem para o A* em diferentes cenários.

2.5.2 Busca A* com Passagem por Farmácias

O algoritmo A* com a restrição de passagem por farmácias obteve uma **taxa de sucesso de 100%**, evidenciando sua eficácia em encontrar caminhos válidos que atendem ao critério de passagem pelas farmácias. As métricas obtidas foram:

- **Custo médio:** 290,855, com um **desvio padrão de 99,55**, indicando uma variabilidade moderada nos custos.
- **Nós gerados (média):** 1421,89, com um **desvio padrão de 1011,53**.
- **Nós visitados (média):** 360,11, com um **desvio padrão de 256,66**.

A distribuição dos custos evidenciou que a presença das farmácias impactou o custo do caminho, aumentando-o em comparação ao A* convencional, mas ainda mantendo um bom nível de consistência nas execuções.

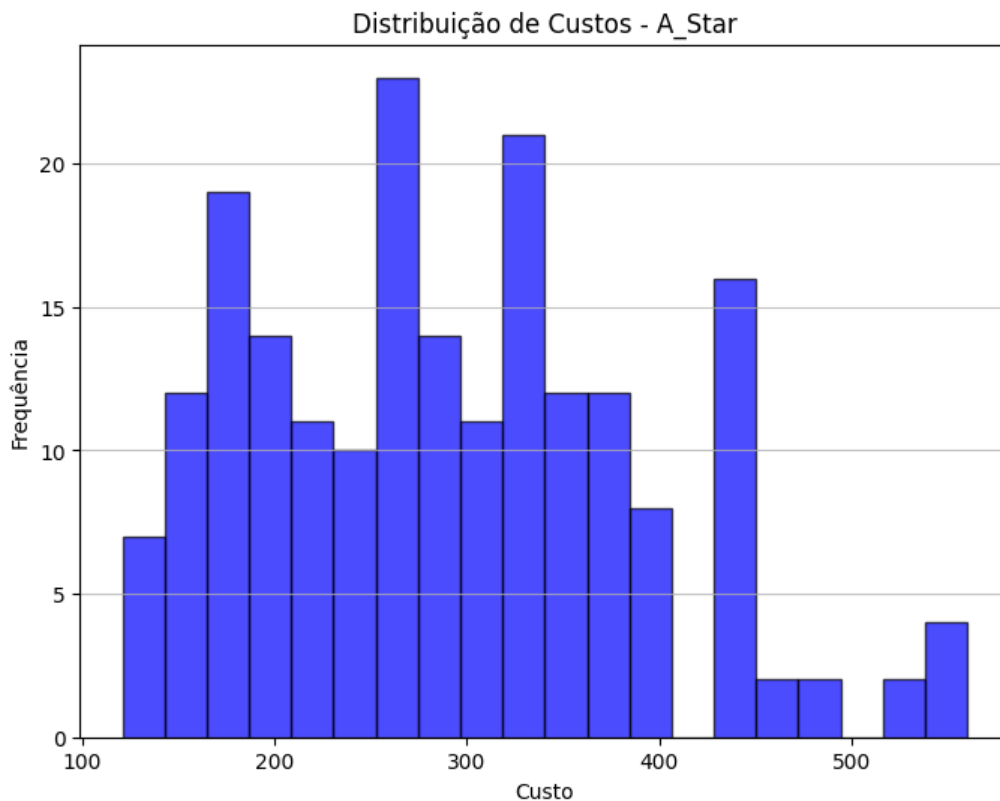


Figura 10: Distribuição dos custos na Busca A* com Passagem por Farmácias.

2.5.3 Comparação com A* Tradicional

- Ambos os algoritmos atingiram uma taxa de sucesso de 100%, confirmando a eficácia do A* em ambos os cenários.
- O custo médio no cenário com farmácias foi maior (290,855) do que no A* tradicional (216,735), o que era esperado devido à necessidade de passagem por pontos específicos (as farmácias).
- O A* com farmácias gerou mais nós, com uma média de 1421,89 nós gerados, em comparação aos 498,76 nós gerados no cenário tradicional. Esse aumento está relacionado ao maior número de restrições a serem atendidas.
- O número de nós visitados também aumentou, com uma média de 360,11 nós visitados no cenário com farmácias, em comparação com 126,49 nós visitados no cenário tradicional. Isso indica uma maior complexidade no processamento devido às restrições adicionais.

Esses resultados destacam o impacto das farmácias no desempenho do algoritmo A*, aumentando o custo do caminho e a quantidade de nós gerados e visitados, mas sem comprometer a taxa de sucesso.